

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Andre Adelfi Stevena - 5024241089

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Blink STM32 Blackpill

1. Buat project PlatformIO untuk Modul 1, lalu isi platformio.ini dengan board blackpill f401cc, framework arduino, dan upload protocol stlink
2. Hubungkan ST-Link ke Blackpill (SWDIO, SWCLK, GND, 3.3V), lalu colokkan ST-Link ke komputer melalui USB
3. Tulis kode blink pada src/main.cpp
4. Klik build dan upload
5. Amati LED yang menyala dan mati

1.2 Percobaan 2: Blink ESP32

1. Ganti isi platformio.ini dengan board esp32dev dan framework espidf
2. Colokkan ESP32 ke laptop
3. Hapus src/main.cpp, lalu buat src/main.c berisi kode blink dengan appmain()
4. Klik Build dan Upload
5. Amati LED berkedip

1.3 Percobaan 3: Pull up dan Pull down

1. Ganti platformio.ini ke framework arduino, hapus src/main.c, lalu buat src/main.cpp
2. Pasang pushbutton 1 pada GPIO 32 dengan resistor pull up eksternal $10\,\mathrm{k}\Omega$ dan pushbutton 2 pada GPIO 33 dengan resistor pull down eksternal $10\,\mathrm{k}\Omega$
3. Upload Kode Program A, tekan kedua tombol, dan catat hasil di Serial Monitor
4. Lepas kedua resistor sehingga tombol tersambung langsung .
5. Upload Kode Program B, tekan kedua tombol, dan catat hasilnya.
6. Bandingkan hasil konfigurasi eksternal dan internal.

1.4 Percobaan 4: Debouncing pada Input Tombol

1. Pasang pushbutton pada GPIO 25 dan GND
2. Tulis kode dengan dua penghitung counterNoDebounce dan counterWithDebounce
3. Upload kode dengan DEBOUNCEDELAY 50 ms, lalu tekan tombol dan bandingkan kedua penghitung di Serial Monitor
4. Ulangi dengan tekanan cepat dan lambat

5. Ubah DEBOUNCEDELAY menjadi 10 ms, upload ulang, lalu uji lagi
6. Ubah menjadi 200 ms, upload ulang, lalu tekan tombol dengan cepat berulang kali.
7. Bandingkan hasil 10 ms, 50 ms, dan 200 ms

1.5 Percobaan 5: Pembacaan ADC dengan LDR

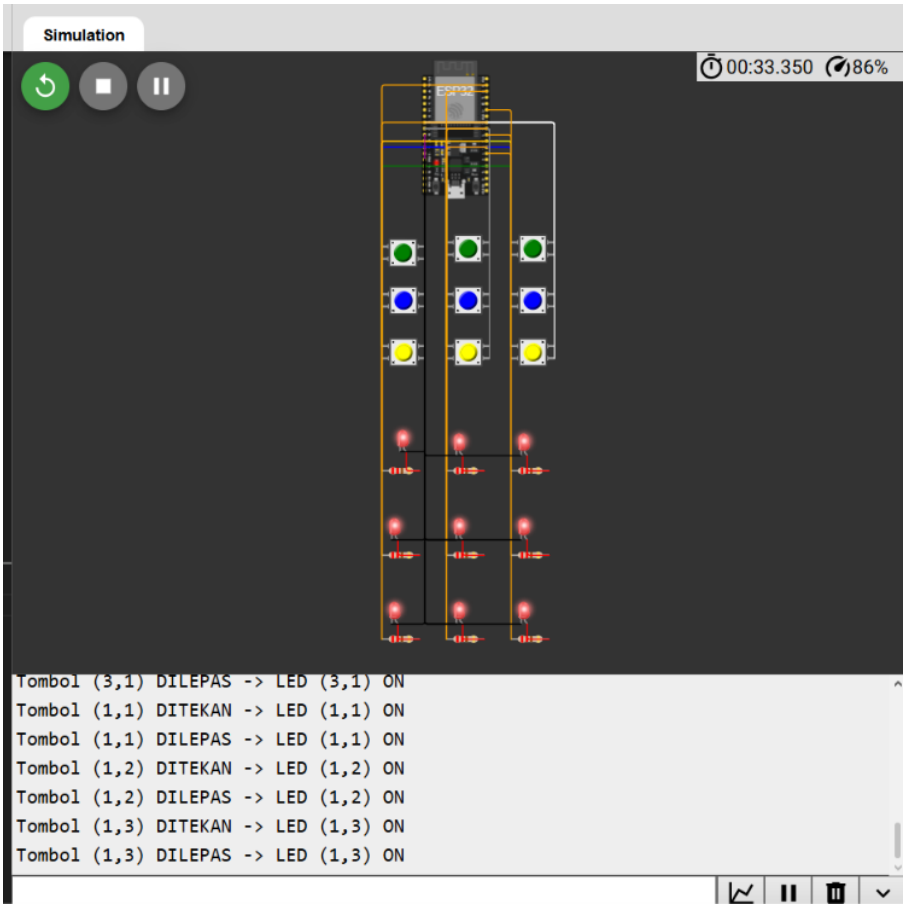
1. Rangkai LDR sebagai pembagi tegangan: satu kaki LDR ke 3.3V, kaki lainnya ke GPIO 34 dan ke GND melalui resistor $10\,\mathrm{k}\Omega$
2. Tulis kode yang membaca analogRead() pada GPIO 34 dan mengubahnya menjadi tegangan
3. Klik build dan upload lalu cek serial monitor
4. Tutup LDR dengan tangan, lalu sinari dengan lampu
5. Catat perubahan nilai ADC (0-4095) dan tegangannya

2 Analisis Hasil Percobaan

pada percobaan 1 Blink STM32 Blackpill menunjukkan ST-Link V2 yang terhubung ke port USB laptop dengan indikator LED merah menyala, serta empat kabel yang tersambung ke header SWD pada Blackpill. Hal ini menandakan jalur pemrograman SWD sudah terpasang sesuai teori, ST-Link berperan sebagai programmer eksternal karena Blackpill tidak memiliki programmer onboard. Keberhasilan percobaan ditandai dengan LED onboard PC13 yang berkedip setiap 500 ms. Karena LED ini aktif LOW, digitalWrite(LEDPIN, LOW) menyalakan LED dan HIGH mematikannya.
pada ercobaan 2 Blink ESP32 dengan ESP-IDF. Log terminal menunjukkan ESP32 terdeteksi pada COM4 dengan chip ESP32-D0WD-V3 dual core 240 MHz, lalu stub flasher dijalankan dan baud rate dinaikkan ke 460800. Ini menandakan komunikasi serial dan proses flashing berjalan. Pada editor terdapat eror “cannot open source file sdkconfig.h” pada include "driver/gpio.h". Peringatan ini berasal dari IntelliSense yang belum memiliki include path lengkap, karena build dan upload tetap berhasil. Secara teori, program ESP-IDF berbeda dari Arduino. Titik masuknya adalah appmain() yang berjalan sebagai task FreeRTOS, pin dikonfigurasi dengan gpiosetdirection(), dan jeda memakai vTaskDelay, yang tidak memblokir task lain, tidak seperti delay().
pada percobaan 3 Pull up dan pull down eksternal vs internal. Pada konfigurasi internal , Serial Monitor menampilkan kondisi “TIDAK DITEKAN” untuk kedua tombol saat tidak ditekan, dan berubah menjadi “DITEKAN” untuk GPIO 32 ketika tombolnya ditekan. Hasil ini sesuai teori pin pull up terbaca HIGH saat terbuka dan LOW saat tombol menghubungkannya ke GND, sehingga logika stateUp = LOW menandakan tekanan. Pull up internal lebih praktis karena tidak membutuhkan komponen tambahan dan mengurangi risiko salah rangkai, tetapi hanya tersedia pada pin tertentu dan nilai resistansinya tidak dapat dipilih.
percobaan 4 Debouncing, pada serial monitor, kedua penghitung bertambah hampir bersamaan setiap tombol ditekan . Selisih satu yang muncul hanyalah efek waktu, karena counterWithDebounce baru bertambah setelah sinyal stabil selama DEBOUNCEDELAY 50 ms, sedangkan counterNoDebounce langsung bertambah pada transisi pertama. Pada ini tidak terlihat penambahan ganda pada

penghitung tanpa debouncing, sehingga bouncing tidak terekam. Hal ini sesuaimodul bahwa bouncing tidak selalu muncul, karena bergantung pada kondisi kontak, kecepatan dan tekanan jari, serta kualitas tombol. Debouncing tetap diperlukan sebagai pengaman. Secara teori, nilai 10 ms dapat terlalu pendek untuk meredam bouncing, 50 ms umumnya cukup, dan 200 ms membuat respons terasa lambat serta berisiko melewatkan tekanan cepat berturut-turut.
pada percobaan 5 ADC dengan LDR, data menunjukkan nilai ADC mentah sekitar 3747-4032 pada kondisi terang, lalu turun ke 3061-3094 dan akhirnya stabil di 2724-2802 ketika LDR ditutup. Ini sesuai teori pada pembagi tegangan dengan LDR ke 3.3V dan resistor 10 k ke GND, cahaya yang lebih terang menurunkan resistansi LDR sehingga tegangan di titik tengah naik, dan sebaliknya. Perubahan nilai yang kontinu ini membuktikan bahwa ADC 12-bit dapat merepresentasikan banyak kondisi (0-4095), tidak hanya HIGH atau LOW seperti input digital.

3 Hasil Tugas Modul



Gambar 1: Hasil simulasi Matrix pushbutton 3x3

Listing 1: Source code

```
const uint8_t ROW_PINS[3] = {13, 14, 27};
const uint8_t COL_PINS[3] = {26, 25, 33};
const uint8_t LED_PINS[3][3] = {
  {23, 22, 21},
  {19, 18,  5},
  {17, 16,  4}
```

```
};

const unsigned long DEBOUNCE_MS = 50;
const bool TOGGLE_MODE = true;
bool lastReading[3][3];
bool stableState[3][3];
unsigned long lastChangeTime[3][3];
bool ledState[3][3];

void scanMatrix(bool raw[3][3]) {
  for (int r = 0; r < 3; r++) {
    for (int i = 0; i < 3; i++) digitalWrite(ROW_PINS[i], HIGH);
    digitalWrite(ROW_PINS[r], LOW);
    delayMicroseconds(50);
    for (int c = 0; c < 3; c++) {
      raw[r][c] = (digitalRead(COL_PINS[c]) == LOW);
    }
  }
  for (int i = 0; i < 3; i++) digitalWrite(ROW_PINS[i], HIGH);
}
void setup() {
  Serial.begin(115200);

  for (int r = 0; r < 3; r++) {
    pinMode(ROW_PINS[r], OUTPUT);
    digitalWrite(ROW_PINS[r], HIGH);
  }
  for (int c = 0; c < 3; c++) pinMode(COL_PINS[c], INPUT_PULLUP);

  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      pinMode(LED_PINS[r][c], OUTPUT);
      digitalWrite(LED_PINS[r][c], LOW);
      lastReading[r][c] = false;
      stableState[r][c] = false;
      lastChangeTime[r][c] = 0;
      ledState[r][c] = false;
    }
  }
  Serial.println("Matrix Pushbutton 3x3 -> Matrix LED siap");
}
void loop() {
  bool raw[3][3];
  scanMatrix(raw);
  unsigned long now = millis();

  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      if (raw[r][c] != lastReading[r][c]) {
        lastChangeTime[r][c] = now;
        lastReading[r][c] = raw[r][c];
      }
      if ((now - lastChangeTime[r][c]) > DEBOUNCE_MS &&
          raw[r][c] != stableState[r][c]) {
```

```
61
62          stableState[r][c] = raw[r][c];
63
64          if (TOGGLE_MODE) {
65            if (stableState[r][c]) ledState[r][c] = !ledState[r][c];
66          } else {
67            ledState[r][c] = stableState[r][c];
68          }
69          digitalWrite(LED_PINS[r][c], ledState[r][c] ? HIGH : LOW);
70
71          Serial.printf("Tombol (%d,%d) %s -> LED (%d,%d) %s\n",
72                        r + 1, c + 1,
73                        stableState[r][c] ? "DITEKAN" : "DILEPAS",
74                        r + 1, c + 1,
75                        ledState[r][c] ? "ON" : "OFF");
76        }
77      }
78    }
79  }
```

1. Debouncing tetap perlu karena bouncing adalah sifat mekanis kontak logam pada pushbutton, bukan akibat banyaknya tombol yang ditekan. Saat ditekan atau dilepas kontak memantul sehingga sinyal berubah HIGH-LOW berulang kali sebelum stabil. Karena fungsi scanMatrix() dipanggil terus-menerus di loop(), ini terbaca sebagai beberapa kali penekanan, sehingga pada mode toggle LED bisa berganti status berkali kali dan hasil akhirnya acak. Dengan DEBOUNCEMS sebesar 50 ms, tombol dan LED baru berubah setelah pembacaan tidak berubah selama waktu itu, jadi satu tekanandihitung satu kejadian.

2. memakai pull up internal ESP32 pada GPIO 26, 25, dan 33, sehingga saat tombol tidak ditekan pin kolom ditarik ke 3,3 V dan terbaca HIGH. Saat tombol ditekan, kolom terhubung ke baris yang sedang dipindai sehingga terbaca LOW, jadi logikanya aktif LOW. Pin tidak boleh dibiarkan floating karena tegangannya tidak terdefinisi dan ditentukan oleh noise atau kopling kapasitif, sehingga bisa terbaca HIGH atau LOW secara acak walaupun tombol tidak disentuh dan LED bisa berubah sendiri.

3. Pada digital input, pin hanya membandingkan tegangan dengan ambang sehingga hasilnya hanya satu bit (HIGH atau LOW), yang cukup untuk tombol yang hanya punya dua keadaan. Pada ADC, tegangan dari LDR berubah kontinu mengikuti intensitas cahaya, lalu diukur dan dikuantisasi menjadi nilai 0-4095 pada ADC 12-bit ESP32. Karena itu ADC dapat merepresentasikan banyak kondisi (gelap, redup, terang, dan seterusnya), tidak hanya dua. Untuk LDR, gunakan pin ADC1 yang masih kosong seperti GPIO 32, bukan pin matriks.

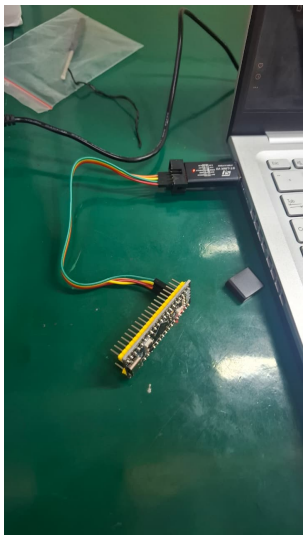
4 Kesimpulan

Praktikum ini berhasil membuktikan secara teori, Percobaan 1 Blink STM32, berhasil membuktikan bahwa Blackpill dapat diprogram lewat ST-Link dengan framework Arduino. LED onboard PC13 aktif LOW, sehingga LOW menyalakan LED dan HIGH mematikannya. Percobaan 2 Blink ESP32 ESP-IDF, berhasil membuktikan bahwa ESP32 dapat diprogram langsung lewat USB dengan struktur appmain() dan vTaskDelay() berbasis FreeRTOS, berbeda dari setup()/loop() dan delay() pada Arduino.

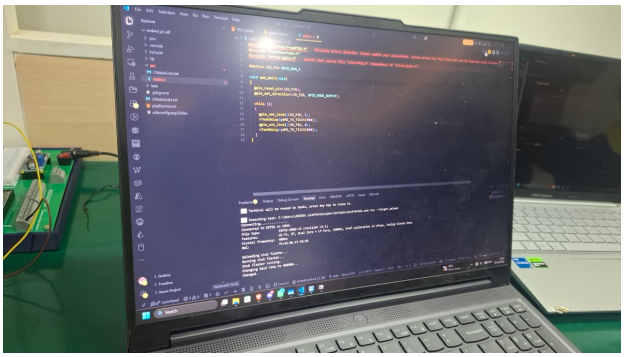
Percobaan 3 Pull-up dan pull-down, pull up internal terbaca HIGH saat tidak ditekan dan LOW saat ditekan, sehingga sesuai teori. Percobaan 4 Debouncing, mekanisme debouncing berbasis millis() berjalan benar dan tetap diperlukan sebagai pengaman. Percobaan 5 ADC dengan LDR, berhasil membuktikan teori nilai ADC berubah kontinu dari sekitar 3747-4032 terang ke sekitar 2724-2802 ditutup, sehingga ADC dapat merepresentasikan banyak kondisi, tidak hanya HIGH dan LOW.

5 Lampiran

5.1 Dokumentasi saat praktikum



(a) ST-Link dan Blackpill

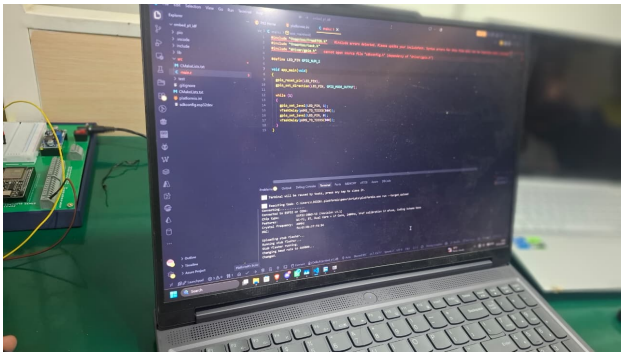


(b) Tampilan

Gambar 2: Dokumentasi Percobaan 1

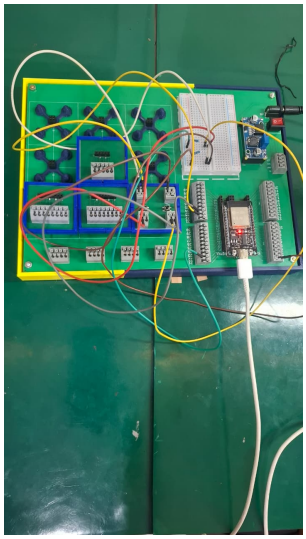


(a) Board ESP32

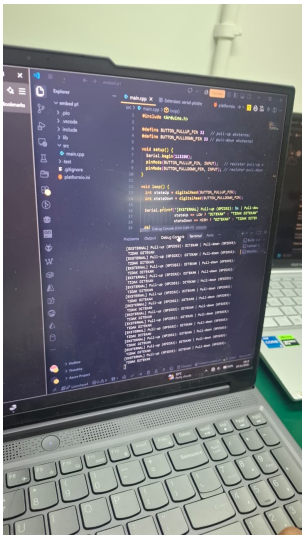


(b) upload ESP-IDF

Gambar 3: Dokumentasi Percobaan 2

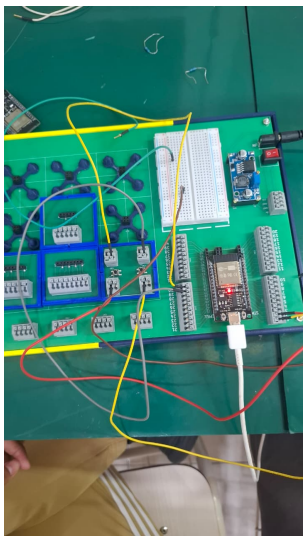


(a) Rangkaian pull-up dan pull-down eksternal

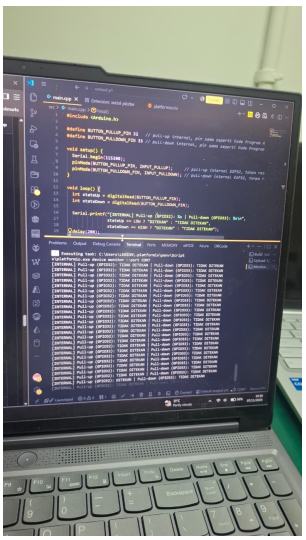


(b) output Serial Monitor

Gambar 4: Dokumentasi Percobaan 3 konfigurasi eksternal

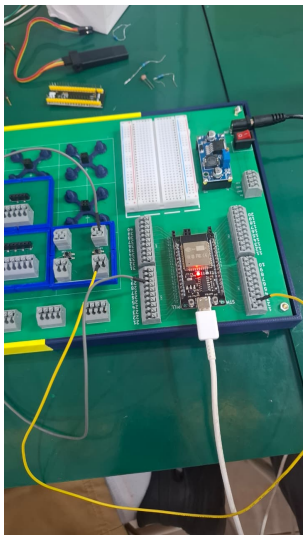


(a) Rangkaian pull up dan pull down internal

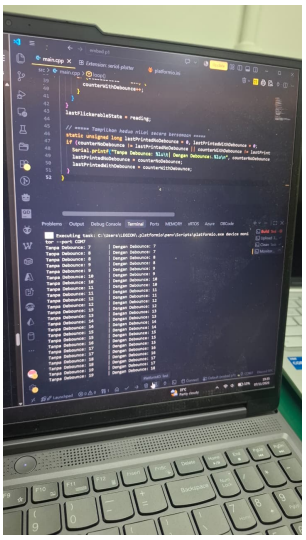


(b) output Serial Monitor

Gambar 5: Dokumentasi Percobaan 3 konfigurasi internal

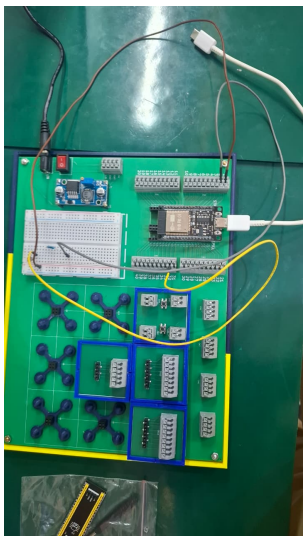


(a) Rangkaian

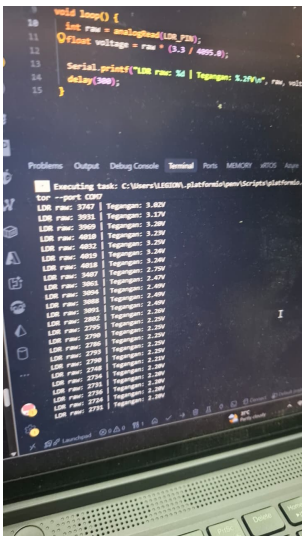


(b) output Serial Monitor

Gambar 6: Dokumentasi Percobaan 4



(a) Rangkaian LDR



(b) output Serial Monitor

Gambar 7: Dokumentasi Percobaan 5